It sounds like you've hit the classic wall of **Proportional-only (P)** control. While a P-controller is great for its simplicity, it inherently struggles with dynamic lag because it only reacts to the *current* error. As the motion speeds up, the system can't "keep up" because it isn't accounting for the momentum of the platform or the rate at which the error is changing.

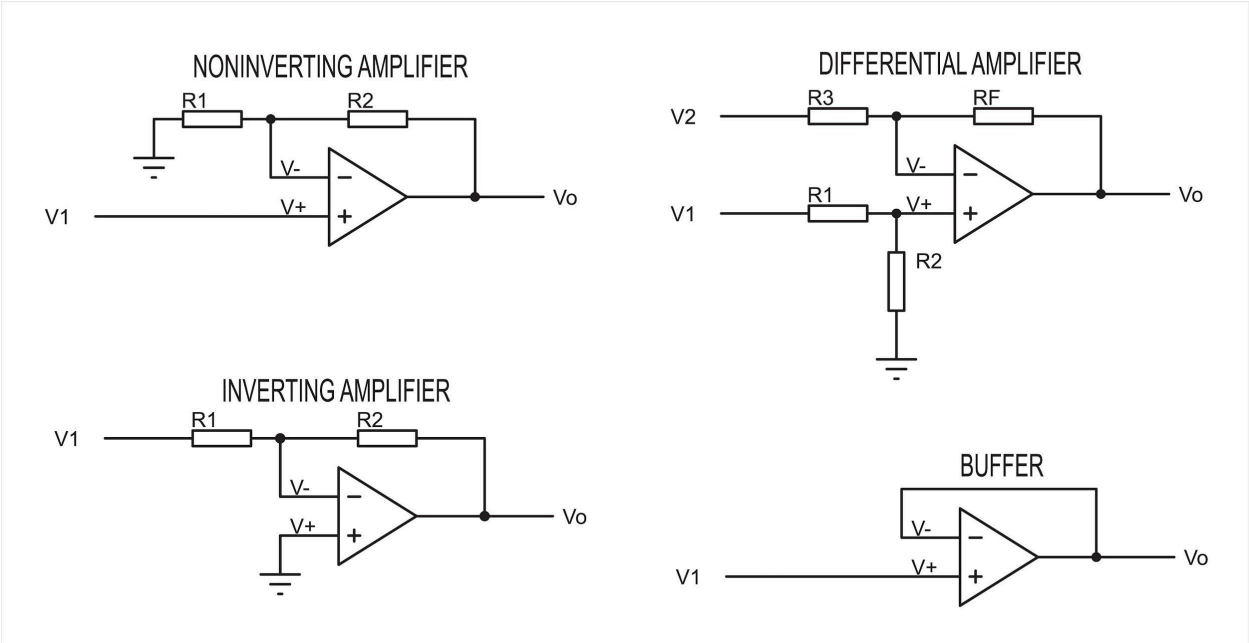
Here is a breakdown of the control solutions to eliminate that lag and sharpen your stabilization.

---

1. Add Derivative Action (Switch to PD Control)

The most direct fix for "lag" in a moving system is adding a **Derivative (D)** term.

- **How it works:** The D-term looks at the *slope* of the error. If the error is increasing rapidly (fast motion), the D-term adds an extra kick to the motor output to "anticipate" where the platform needs to be.
- **The Result:** It acts like a damper. It reduces overshooting and allows you to increase your P-gain higher than before without the system becoming unstable.



[Opens in a new window](#) Shutterstock

---

2. Implement Feedforward Control

If your landing platform is receiving target coordinates from a sensor (like a camera or GPS tracking the drone), you likely know the **target velocity**.

- **The Problem:** A feedback controller (P or PD) must wait for an error to exist before it reacts.
- **The Solution:** Use **Feedforward (FF)**. You pass the target's known velocity directly to your motor output, scaled by a feedforward gain (Kff).
- **The Logic:** The Feedforward term does 90% of the heavy lifting to match the drone's speed, while the P-controller just cleans up the small remaining 10% of positioning error.

$$Output = (Kp \cdot Error) + (Kd \cdot dtdError) + (Kff \cdot TargetVelocity)$$

---

3. Check for Mechanical & Signal Latency

Sometimes the "lag" isn't in the math, but in the hardware. Before rewriting your code, check these three bottlenecks:

- **Loop Frequency:** If your control loop is running at 50Hz but the drone is moving fast, the platform is "blind" between updates. Try to push your loop rate to **200Hz - 400Hz**.
- **Sensor Filtering:** Are you using a heavy low-pass filter on your sensor data? Excessive filtering cleans up noise but introduces phase lag (delay). You might need a **Kalman Filter** or a more aggressive complementary filter to get cleaner data without the delay.
- **Actuator Deadband:** If your motors don't move until the PWM signal reaches a certain threshold, the platform will "wait" at every direction change. Ensure your code compensates for the motor's minimum starting voltage.

Summary of Improvements

Solution	Difficulty	Impact on Lag
Increase P-Gain	Easy	Minimal (eventually causes oscillation)
Add D-Term (PD)	Medium	High (adds damping and anticipation)
Feedforward	Medium	Very High (removes delay in tracking)
Higher Loop Rate	Harder	Critical (reduces overall system latency)

Export to Sheets

Are you currently calculating your velocity error based on a vision system (like ArUco tags) or is it purely IMU-based?